

Xiaozhi AI Robot

Tài liệu tích hợp & kế hoạch tùy biến

ESP32-S3 + ES3C28P + 3× Servo MG996R

+ WebSocket Server tự host

Tài liệu kỹ thuật nội bộ

2026-05-10

Table of Contents

Mục đích & phạm vi	3
Bài toán	3
Mục tiêu của tài liệu.....	3
Khái niệm nền tảng	3
ESP32-S3 là gì?	3
ESP-IDF là gì?	4
Firmware là gì?	4
Build pipeline là gì?	5
Default config là gì?	5
Bootloader là gì?	6
Partition table là gì?	6
NVS (Non-Volatile Storage) là gì?	6
WebSocket là gì & hoạt động sao?	6
Quy trình xiaozhi dùng WebSocket.....	7
MCP (Model Context Protocol) là gì?	8
Code mẫu MCP tool (lấy từ board ESP-Hi robot dog).....	8
AFE (Audio Front-End) & Wake Word.....	8
Phần cứng — board ES3C28P	9
Tổng quan	9
Linh kiện chính	9
Pin assignment chính thức.....	9
Pad hàn ở mặt sau board.....	11

Kế hoạch dây servo MG996R	11
Bản đồ source code xiaozhi-esp32	11
Cấu trúc cấp gốc.....	11
Folder docs/ — đọc trước khi code	12
Folder main/ — chi tiết.....	12
File cấp gốc.....	12
Folder boards/	12
Folder audio/	13
Folder display/	13
Folder protocols/ — chỗ tích hợp WebSocket custom	13
Folder assets/	13
Cơ chế chọn protocol & kế hoạch switch sang server custom	14
Cơ chế hiện tại	14
Spec OTA response server custom cần trả	14
3 cách switch	15
Kế hoạch tích hợp servo + MCP	15
Cần code thêm gì?	15
MCP tool đăng ký	16
Phía LLM (server tự host) — prompt nên có	16
Workflow build, flash, test.....	17
Build	17
Flash (Flash Download Tool)	17
Manual bootloader entry (nếu fail)	17
Monitor serial sau khi flash.....	17
Plan tổng — chia phase.....	18
Câu hỏi cần xác nhận trước khi code.....	18
Glossary — từ điển	18
Tài liệu tham khảo.....	20

Tài liệu này map toàn bộ source code dự án xiaozhi-esp32, giải thích các khái niệm cốt lõi (build pipeline, ESP-IDF, MCP, WebSocket, AFE wake word), liệt kê pin GPIO còn rảnh trên board ES3C28P, và đề xuất kế hoạch tích hợp:

- Tự host WebSocket backend (đã có người làm) thay thế xiaozhi.me cloud.

- Thêm 3 servo MG996R (đầu, tay, chân) qua MCP tool — LLM gọi xuống device.
- Build + flash + iterate.

Văn bản dùng cho cả người triển khai (con người) và Claude (AI assistant) làm tài liệu tham chiếu.

Mục đích & phạm vi

Bài toán

Bạn có:

- Board phần cứng **ES3C28P / ES3N28P** (LCD wiki) với chip ESP32-S3, màn 2.8' ILI9341 240×320, codec ES8311 (mic + loa), pin sạc.
- Source code xiaozhi-esp32 (C++, ESP-IDF) — assistant AI bằng giọng nói, dùng MQTT/WebSocket để giao tiếp cloud.
- Một WebSocket server tự host (do người khác viết, gọi API Google) — sẽ thay thế `api.tenclass.net`.
- Kế hoạch lắp 3 servo MG996R làm robot có chuyển động (đầu xoay, tay vẫy, chân di chuyển).

Mục tiêu của tài liệu

1. Cung cấp **từ điển khái niệm** các thuật ngữ ESP-IDF / firmware / IoT.
2. Chỉ rõ **vai trò từng folder** trong source code xiaozhi.
3. Liệt kê **pin GPIO đang dùng & còn rảnh** (trích từ tài liệu chính thức của kit).
4. Đề xuất **kế hoạch tích hợp servo + WebSocket custom** từng bước.
5. Cung cấp **workflow build/flash/test** có thể lặp lại.

Khái niệm nền tảng

ESP32-S3 là gì?

ESP32-S3 là vi điều khiển hệ thống-trên-chip (SoC) của Espressif, kiến trúc Xtensa LX7 dual-core 32-bit @ 240 MHz, tích hợp Wi-Fi 802.11 b/g/n và Bluetooth LE 5.0. Phiên bản dùng cho board này là ESP32-S3-WROOM-1, có:

- 16 MB Flash QSPI (lưu firmware + assets).

- 8 MB PSRAM Octal (RAM mở rộng — dùng cho LVGL framebuffer + audio buffer + AI model).
- 45 GPIO (nhưng không phải tất cả đều exposed trên kit ES3C28P — xem mục 3).
- Hardware accelerator cho I2S, USB JTAG/Serial native, AES, SHA, RSA.

ESP-IDF là gì?

ESP-IDF (Espressif IoT Development Framework) là framework chính thức để lập trình các chip ESP32 (S3, C3, P4, ...). Nó cung cấp:

- Trình biên dịch chuyên dụng (`xtensa-esp-elf-gcc`).
- Bootloader 2 stage + partition table.
- FreeRTOS làm nhân hệ điều hành.
- Driver cho tất cả ngoại vi (I2C, I2S, SPI, USB, Wi-Fi, BLE).
- Component manager (giống npm/pip) để tải driver bên thứ 3.

Phiên bản đang dùng: ESP-IDF v5.5.4 — cài qua **EIM** (ESP-IDF Installation Manager) tại `C:\esp\v5.5.4\esp-idf`.

Firmware là gì?

Firmware = phần mềm chạy trên vi điều khiển. Khác app PC ở chỗ:

- Không có hệ điều hành đầy đủ (Linux/Windows). Thay vào là **FreeRTOS** (hệ điều hành nhẹ, real-time, dạng task scheduler).
- Khi flash xong, chip chạy ngay không cần *install*.
- Toàn bộ logic, driver, asset (ảnh, âm thanh) đóng gói thành **1 file .bin** ghi vào Flash.

Trong dự án xiaozhi, file output cuối là `build/merged-binary.bin` (~10.4 MB), chứa:

1. Bootloader (`0x0000`)
2. Partition table (`0x8000`)
3. OTA data (`0xD000`)
4. App chính `xiaozhi.bin` (`0x20000`)
5. Assets (`0x800000`) — chứa file `.ogg` + `language.json`

Build pipeline là gì?

Build pipeline = chuỗi các bước biến source code (C/C++) thành file firmware sẵn sàng nạp vào chip. Trong xiaozhi, pipeline được tự động hóa qua `scripts/release.py`:

1. **Set target:** `idf.py set-target esp32s3` — chỉ định chip đích, sinh `sdkconfig` từ `sdkconfig.defaults`.
2. **Áp dụng board config:** đọc `boards/<board>/config.json`, append các `CONFIG_...=y` vào `sdkconfig`.
3. **Resolve dependencies:** ESP-IDF Component Manager đọc `idf_component.yml`, tải các thư viện cần (LVGL, esp-sr, esp-codec-dev...) vào `managed_components/`.
4. **CMake configure:** sinh `build.ninja` từ `CMakeLists.txt`.
5. **Compile:** `ninja` biên dịch ~1500 file `.c/.cc` → `.o` → link thành `xiaozhi.elf`.
6. **Convert .elf → .bin:** `esptool.py elf2image`.
7. **Generate assets partition:** nén `main/assets/locales/<lang>/*.ogg` thành `generated_assets.bin`.
8. **Merge bin:** gộp 5 file (bootloader, partition table, ota data, xiaozhi, assets) thành 1 file duy nhất `merged-binary.bin`.
9. **Zip:** đóng gói vào `releases/v<version>_<board>.zip`.

xiaozhi.me cloud dùng **MQTT broker** làm trục giao tiếp chính (lý do: scale tốt cho hàng ngàn device cùng lúc). Tuy nhiên với server tự host, đa số dev ưa **WebSocket** hơn vì đơn giản, không cần broker, dùng port 443/80 để xuyên firewall. Project xiaozhi hỗ trợ cả 2 — chọn protocol động dựa trên config server trả về.

Default config là gì?

Default config = giá trị mặc định cho hàng trăm setting của ESP-IDF (kích thước heap, log level, partition layout, Wi-Fi buffer, ...). Trong xiaozhi có nhiều cấp:

- `sdkconfig.defaults` — chung cho mọi target.
- `sdkconfig.defaults.esp32s3` — riêng cho chip S3 (vd. `CONFIG_SPIRAM_MODE_OCT=y` bật octal PSRAM).
- `boards/<board>/config.json` — riêng cho board (vd. `CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y`).
- Khi chạy `idf.py menuconfig` — user override interactive.

Khi build, ESP-IDF merge tất cả file trên thành `sdkconfig` cuối cùng (`build/config/sdkconfig.h`).

Bootloader là gì?

Bootloader = đoạn code nhỏ (~30 KB) chạy đầu tiên khi chip cấp điện. Chức năng:

1. Khởi tạo clock, PSRAM, flash.
2. Đọc **partition table** để biết app nằm ở đâu (ota_0 hoặc ota_1).
3. Verify checksum/signature của app.
4. Nhảy vào app entry point.

Nếu app corrupt (vd. flash hỏng giữa chừng), bootloader sẽ **reset chip** → bootloop. Đó là lý do hôm trước khi esptool flash failed 50%, board bootloop liên tục.

Partition table là gì?

Partition table = bảng phân bổ vùng nhớ trên Flash 16MB:

```
[language=, caption={partitions/v2/16m.csv (rút gọn)}]
# Name,      Type, SubType,  Offset,   Size
nvs,         data, nvs,        0x9000,   0x4000   # WiFi creds, settings
otadata,     data, ota,        0xd000,   0x2000   # OTA slot indicator
phy_init,    data, phy,        0xf000,   0x1000   # WiFi RF calibration
ota_0,       app,  ota_0,         0x20000,  0x7E0000 # ~8MB app slot 1
ota_1,       app,  ota_1,         0x80000,  0x7E0000 # ~8MB app slot 2 (OTA)
assets,      data, spiffs,     0xFE000,  0x20000  # voice .ogg + language strings
```

2 slot ota_0 và ota_1 cho phép update OTA (Over-The-Air): chip ghi app mới vào slot rảnh, verify, switch active slot, reboot. Nếu fail → fallback slot cũ.

NVS (Non-Volatile Storage) là gì?

NVS = key-value store nhỏ (16 KB) trên flash, lưu các thông tin cần persist qua reboot:

- Wi-Fi SSID + password.
- Server URL (sau khi nhận từ OTA endpoint).
- User preferences (volume, brightness).
- Activation token, device ID.

Trong xiaozhi, đọc/ghi NVS qua class Settings (main/settings.cc). Khi erase flash → mất hết → board phải config lại Wi-Fi từ đầu.

WebSocket là gì & hoạt động sao?

WebSocket (RFC 6455) = giao thức truyền tin 2 chiều (full-duplex) trên port HTTP/HTTPS. Khác HTTP thông thường ở chỗ:

- HTTP: client gửi request, server trả response, đóng kết nối.
- WebSocket: client gửi 1 request HTTP đặc biệt (Upgrade: websocket), server chấp nhận → kết nối TCP **giữ mở vĩnh viễn** → 2 phía gửi tin nhắn bất cứ lúc nào.

Quy trình xiaozhi dùng WebSocket

1. Device kết nối Wi-Fi → có IP.
2. Device gọi POST `https://server.com/ota` với header Device-Id, Client-Id.
3. Server trả JSON, trong đó có `websocket_url` (vd. `wss://server.com/ws`).
4. Device mở WebSocket tới URL đó, header có Authorization: Bearer <token>, Protocol-Version: 1.
5. Device gửi hello JSON (audio params).
6. Server reply hello (sample rate, session_id).
7. Khi user nói "Hi, ESP" → device wake → bắt đầu gửi **audio Opus binary** qua WS.
8. Server STT → LLM → TTS → gửi **audio Opus binary** ngược về.
9. Song song: text JSON cho TTS state, MCP commands.

```
{
  "type": "hello",
  "version": 1,
  "features": { "mcp": true, "aec": true },
  "transport": "websocket",
  "audio_params": {
    "format": "opus",
    "sample_rate": 16000,
    "channels": 1,
    "frame_duration": 60
  }
}

{
  "type": "hello",
  "transport": "websocket",
  "session_id": "xxx",
  "audio_params": {
    "format": "opus",
    "sample_rate": 24000,
    "channels": 1,
    "frame_duration": 60
  }
}
```

MCP (Model Context Protocol) là gì?

MCP = giao thức chuẩn cho AI gọi **tool** ở thiết bị/server. Anthropic công bố cuối 2024, đang thành chuẩn industry. Trong xiaozhi:

- Device đăng ký **tool** (như set_volume, take_photo).
- Backend gọi tools/list → device trả schema (tên, mô tả, params).
- LLM (Qwen/DeepSeek) đọc schema, quyết định gọi tool nào khi cần.
- Backend gọi tools/call với args → device thực thi → trả kết quả.

Code mẫu MCP tool (lấy từ board ESP-Hi robot dog)

```
void InitializeTools() {
    auto& mcp_server = McpServer::GetInstance();

    mcp_server.AddTool("self.dog.basic_control",
        "Robot basic motion: forward/backward/left/right/idle",
        PropertyList({
            Property("action", kPropertyTypeString)
        }),
        [this](const PropertyList& args) -> ReturnValue {
            std::string action = args["action"].value<std::string>();
            if (action == "forward") {
                servo_dog_ctrl_send(DOG_STATE_FORWARD, NULL);
            } else if (action == "backward") {
                servo_dog_ctrl_send(DOG_STATE_BACKWARD, NULL);
            }
            return true;
        });
}
```

AFE (Audio Front-End) & Wake Word

AFE = Espressif's audio pipeline gồm AEC (echo cancel) + NS (noise suppress) + VAD (voice activity detect) + WakeNet (wake word detection). Chạy trên chip S3 không cần cloud.

Wake word hiện tại: wn9s_hiesp (nhỏ, "Hi, ESP" English) + wn9_hiesp (chuẩn). Không có wake word offline cho tiếng Việt — Espressif chưa ra. Phải dùng "Hi, ESP" hoặc "Nǐ hǎo, xiǎo zhì" (你好小智).

Phần cứng — board ES3C28P

Tổng quan

Board của bạn từ LCD wiki (https://www.lcdwiki.com/2.8inch_ESP32-S3_Display) — kit ES3C28P (touch capacitive) hoặc ES3N28P (touch resistive / không touch). Match 100% với freenove-esp32s3-display-2.8-lcd trong xiaozhi project.

Linh kiện chính

- Chip: ESP32-S3-WROOM-1-N16R8 (16MB Flash, 8MB PSRAM)
- Màn: ILI9341V 2.8" IPS 240×320, SPI 4-line
- Touch: FT6336G I2C (chỉ trên ES3C28P)
- Audio codec: ES8311 (mono, mic + DAC)
- Mic: MEMS LMA2718B381-OA7
- Loa amp: FM8002E (class-D)
- LDO: ME6217 (3.3V)
- Sạc pin: TP4054 (Li-ion 1S)
- LED RGB: WS2812B

Pin assignment chính thức

Trích từ ESP32-S3 芯片 IO 资源分配表.xlsx của kit. Cột "Exposed" = có pad hàn được trên PCB.

Pin assignment ES3C28P (nguồn: kit official xlsx)

GPIO	Exposed?	Đang dùng cho	Có thể dùng IO?
0	N	BOOT button	X
1	N	PA enable (loa)	X
2	Y	—	✓ FREE
3	Y	—	✓ FREE
4	N	I2S MCLK	X
5	N	I2S BCLK	X
6	N	I2S DIN (mic)	X
7	N	I2S WS / LRCK	X
8	N	I2S DOUT (loa)	X

GPIO	Exposed?	Đang dùng cho	Có thể dùng IO?
9	N	ADC8 (battery)	X
10	N	LCD CS	X
11	N	LCD MOSI	X
12	N	LCD SCK	X
13	N	LCD MISO	X
14	Y	—	✓ FREE
15	Y	I2C SCL (codec/touch)	△ chỉ I2C
16	Y	I2C SDA (codec/touch)	△ chỉ I2C
17	N	Touch INT	X
18	N	Touch RST	X
19	N	USB D-	X
20	N	USB D+	X
21	Y	—	✓ FREE
26-37	N	Internal flash/PSRAM	X (cấm tuyệt đối)
38-41	N	SD card SDIO (chưa hàn)	X
42	N	RGB LED WS2812B	X
43	Y	UART0 TX (debug)	△ nếu không dùng UART
44	Y	UART0 RX (debug)	△ nếu không dùng UART
45	N	LCD backlight	X
46	N	LCD DC	X
47	N	SD SDIO DATA3	X
48	N	SD SDIO DATA2	X

GPIO 0, 3, 45, 46 là **strap pin** — quyết định boot mode lúc reset. Sau khi boot xong, có thể dùng làm IO bình thường, nhưng **trạng thái lúc cấp điện** ảnh hưởng:

- GPIO 0 = LOW lúc reset → vào download mode (dùng cho flash).
- GPIO 3 = float lúc reset (không kéo lên/xuống ngoài).
- GPIO 45, 46 = liên quan voltage regulator config.

Khi nối servo vào GPIO 2 hay 3, đảm bảo dây tín hiệu từ board tới servo có điện trở pull-down 10k để tránh ảnh hưởng strap. Hoặc đơn giản: **ưu tiên GPIO 14 và 21 trước**, GPIO 2/3 chỉ dùng nếu thiếu.

Pad hàn ở mặt sau board

Theo bảng IO chính thức, **board có sẵn pad hàn (是否引出 = Y)** cho các chân: **GPIO 2, 3, 14, 15, 16, 21, 43, 44**. Xem schematic PDF [2.8inch_ESP32-S3_Display_Schematic.pdf](#) để xác định vị trí vật lý mỗi pad.

Kế hoạch dây servo MG996R

Phân chia chân:

Servo	GPIO	Lý do
Đầu (Head)	14	Hoàn toàn rảnh, không strap, gần khu vực vào USB
Tay (Arm)	21	Hoàn toàn rảnh, không strap
Chân (Leg)	2	Strap pin nhưng OK sau boot, exposed

Cấp nguồn servo MG996R:

- Mỗi servo MG996R cần 4.8–7.2V, dòng peak ~1.5–2A.
- 3 servo đồng thời = up to 6A peak.
- Tuyệt đối không lấy từ pin board.** Cấp nguồn riêng:
 - Pin Li-ion 2S (7.4V) → buck converter → 6V cấp servo.
 - HOẶC pin Li-ion 1S (3.7V) → boost converter → 5V/6A cấp servo.
 - Tách **GND chung** với ESP32 board (mass chung), **KHÔNG** nối Vcc chung.
- Dây tín hiệu PWM từ GPIO của ESP32 → trực tiếp vào servo signal pin (không cần level shifter — servo thấy 3.3V là logic HIGH).
- Nên có tụ **1000µF** ở đầu Vcc của mỗi servo để hấp thu dòng inrush.

Bản đồ source code xiaozhi-esp32

Cấu trúc cấp gốc

```
xiaozhi-esp32-main/
|-- CMakeLists.txt          # version 2.2.6, MINIMAL_BUILD on
|-- docs/                  # documentation chính thức
|-- main/                  # source code (chính)
|-- managed_components/    # deps tu tai (LVGL, esp-sr, ...)
|-- partitions/v2/16m.csv  # partition table 16MB
|-- scripts/release.py     # build pipeline
|-- sdkconfig              # ESP-IDF config (auto-generated)
|-- sdkconfig.defaults     # default cho moi target
|-- sdkconfig.defaults.esp32s3 # default cho chip S3
```

```
| -- build/merged-binary.bin # output cuoi (10.4MB)
`-- releases/v2.2.6_*.zip    # zip release
```

Folder `docs/` — đọc trước khi code

I|Y File & Nội dung

`websocket.md` & Spec WebSocket protocol — quan trọng cho integrate server
`mqtt-udp.md` & Spec MQTT (default xiaozhi.me)
`mcp-protocol.md` & Spec MCP
`mcp-usage.md` & Hướng dẫn viết MCP tool — quan trọng cho servo
`custom-board.md` & Cách thêm board mới
`code_style.md` & Convention code

Folder `main/` — chi tiết

File cấp gốc

I|Y File & Vai trò

`main.cc` & Entry point, gọi `app_main()`
`application.cc/h` & Singleton chính. Init mọi thứ. Vòng lặp event chính.
`device_state.h` & Enum DeviceState (idle, connecting, listening, speaking, ...)
`device_state_machine.cc/h` & State transitions với guard
`settings.cc/h` & Đọc/ghi NVS (Wi-Fi, server URL, prefs)
`ota.cc/h` & Gọi ota endpoint, NHẬN URL websocket/mqtt từ server
`system_info.cc/h` & UUID, MAC, free heap, version
`mcp_server.cc/h` & MCP singleton — đăng ký tool cho LLM gọi
`assets.cc/h` & Load file .ogg + language strings từ flash partition
`idf_component.yml` & Khai báo dependency (lvgl, esp-sr, ...)
`Kconfig.projbuild` & Menu config (chọn board, lang, wake-word)
`CMakeLists.txt` & Build rules + register source

Folder `boards/`

Mỗi board 1 folder. Folder của bạn:

```
boards/freenove-esp32s3-display-2.8-lcd/
|-- config.h          # GPIO pinout
|-- config.json       # build config (LANG=VI_VN đang dùng)
|-- freenove-esp32s3-display-2.8-lcd.cc # class init board
`-- ReadMe.md
```

Folder `boards/common/` — class chung dùng được cho mọi board:

I|Y File & Vai trò

`board.cc/h` & Base class Board

wifi_board.cc/h & Board WiFi-only (kế thừa Board)
button.cc/h & Driver nút bấm (GPIO + click/long-press)
backlight.cc/h & PWM cho LCD backlight
adc_battery_monitor.cc/h & Đo pin qua ADC
ml307_board.cc/h & Board 4G (cho board dùng modem ML307)

Board esp-hi/ — robot dog mẫu, có 11 MCP tool điều khiển servo. Sẽ copy logic này khi làm robot 3-servo.

Folder audio/

||Y File/folder & Vai trò

audio_codec.cc/h & Base class codec
audio_service.cc/h & I2S read/write loop, pipeline orchestration
codecs/es8311_audio_codec.cc/h & Driver ES8311 (đang dùng)
codecs/no_audio_codec.cc & Cho board không codec (raw I2S)
wake_words/afe_wake_word.cc/h & Espressif AFE pipeline
processors/audio_debugger.cc & Gửi raw audio qua UDP để debug
demuxer/ogg_demuxer.cc & Parse file .ogg

Folder display/

||Y File/folder & Vai trò

display.cc/h & Base class
lcd_display.cc/h & LCD SPI (board của bạn dùng)
oled_display.cc/h & OLED I2C (board nhỏ)
emote_display.cc/h & Display có animation (esp-hi)
lvgl_display/lvgl_theme.cc/h & Theme dark/light, màu sắc
lvgl_display/lvgl_font.cc/h & Font register
lvgl_display/emoji_collection.cc/h & List emoji
lvgl_display/gif/, jpg/ & Decoder ảnh

Folder protocols/ — chỗ tích hợp WebSocket custom

```
protocols/  
|-- protocol.cc/h           # base abstract class  
|-- mqtt_protocol.cc/h      # MQTT impl (xiaozhi.me dung)  
`-- websocket_protocol.cc/h # WebSocket impl - DUNG CHO SERVER CUA BAN
```

Folder assets/

```
assets/  
|-- lang_config.h           # include language tương ứng  
|-- common/                 # .ogg dung chung (popup, success, ...)  
`-- locales/<lang>/         # .ogg + language.json
```

```
|-- vi-VN/          # TIENG VIET (dang dung)
|-- en-US/, zh-CN/, ...
```

Cơ chế chọn protocol & kế hoạch switch sang server custom

Cơ chế hiện tại

File main/application.cc dòng 473–487:

```
void Application::InitializeProtocol() {
    if (ota_>HasMqttConfig()) {
        protocol_ = std::make_unique<MqttProtocol>();
    } else if (ota_>HasWebsocketConfig()) {
        protocol_ = std::make_unique<WebsocketProtocol>();
    } else {
        ESP_LOGW(TAG, "No protocol specified, using MQTT");
        protocol_ = std::make_unique<MqttProtocol>();
    }
    // ... attach callbacks ...
}
```

Logic:

1. Device gọi OTA URL (mặc định `https://api.tenclass.net/xiaozhi/ota/`).
2. Server trả JSON config có thể chứa: `mqtt_endpoint` HOẶC `websocket_url`.
3. Device chọn protocol tương ứng.
4. Lưu config vào NVS để dùng cho session sau.

Spec OTA response server custom cần trả

T cần đọc kỹ main/ota.cc để chốt format chính xác. Gợi ý sơ:

```
{
  "server_time": { "timestamp": 1728000000000, "timezone_offset": 420 },
  "firmware": {
    "version": "2.2.6",
    "url": ""
  },
  "websocket": {
    "url": "wss://your-server.com/ws/xiaozhi",
    "version": 1,
    "token": "your-bearer-token"
  },
  "activation": {
    "code": "123456",
  }
```

```
    "message": "Goto https://your-server.com to activate"  
  }  
}
```

3 cách switch

1. **Đổi OTA URL trở tới server custom** (sạch nhất). Sửa `Kconfig.projbuild` → `CONFIG_OTA_URL`. Server custom phải implement endpoint `/ota`.
2. **Hard-code WebSocket URL**, bỏ qua OTA. Sửa `application.cc` luôn dùng `WebSocketProtocol`, đọc URL từ `Settings (NVS)` thay vì OTA response.
3. **Proxy OTA**. Server custom đứng giữa, fetch OTA gốc từ `xiaozhi.me`, override field `websocket_url`. Không cần sửa firmware.

Khuyến nghị: Cách 1. Vì:

- Đã có người làm WebSocket server.
- Chỉ cần thêm 1 endpoint `/ota` trả JSON.
- Firmware sạch, dễ chuyển server sau.

Kế hoạch tích hợp servo + MCP

Cần code thêm gì?

1. File mới: `main/boards/freenove-esp32s3-display-2.8-lcd/servo_controller.cc/h`
 - Wrapper LEDC peripheral cho 3 channel PWM 50Hz.
 - Convert angle (-90 → +90) ra pulse width (0.5ms → 2.5ms = duty 2.5% → 12.5%).
 - API: `ServoController::SetAngle(int channel, int angle)`.
2. Sửa `freenove-esp32s3-display-2.8-lcd.cc`:
 - Thêm member `ServoController servos_;`.
 - Init trong constructor.
 - Thêm hàm `InitializeTools()` đăng ký MCP tool.
3. Sửa `config.h`:
 - `#define SERVO_HEAD_PIN GPIO_NUM_14`
 - `#define SERVO_ARM_PIN GPIO_NUM_21`
 - `#define SERVO_LEG_PIN GPIO_NUM_2`

4. Sửa main/CMakeLists.txt thêm source mới.

MCP tool đăng ký

```
void InitializeTools() {
    auto& mcp = McpServer::GetInstance();

    mcp.AddTool("self.robot.set_head_angle",
        "Quay dau robot. angle: -90 (trai) den 90 (phai), 0 = giua",
        PropertyList({
            Property("angle", kPropertyTypeInteger, -90, 90)
        }),
        [this](const PropertyList& p) -> ReturnValue {
            servos_.SetAngle(SERVO_CH_HEAD, p["angle"].value<int>());
            return true;
        });

    mcp.AddTool("self.robot.wave_arm",
        "Vay tay 3 lan",
        PropertyList(),
        [this](const PropertyList&) -> ReturnValue {
            for (int i = 0; i < 3; i++) {
                servos_.SetAngle(SERVO_CH_ARM, 30); vTaskDelay(pdMS_TO_TICKS
(300));
                servos_.SetAngle(SERVO_CH_ARM, 90); vTaskDelay(pdMS_TO_TICKS
(300));
            }
            return true;
        });

    mcp.AddTool("self.robot.kick_leg",
        "Da chan 1 lan",
        PropertyList(),
        [this](const PropertyList&) -> ReturnValue {
            servos_.SetAngle(SERVO_CH_LEG, 60); vTaskDelay(pdMS_TO_TICKS(40
0));
            servos_.SetAngle(SERVO_CH_LEG, 0);
            return true;
        });
}
```

Phía LLM (server tự host) — prompt nên có

Ban la robot voi 3 servo: dau (xoay trai-phai), tay (vay), chan (da).

Khi nguoi dung yeu cau hanh dong vat ly, hay goi MCP tool tuong ung:

- "nhin sang trai/phai" -> self.robot.set_head_angle(angle)
- "vay tay/chao" -> self.robot.wave_arm
- "da/dam chan" -> self.robot.kick_leg

Workflow build, flash, test

Build

Trên máy Windows này, ESP-IDF v5.5.4 đã cài tại C:\esp\v5.5.4\esp-idf. Wrapper bat đã tạo: idf_env.bat.

```
cd E:\downloads\xiaozhi-esp32-main\xiaozhi-esp32-main
cmd.exe /c "idf_env.bat python scripts\release.py freenove-esp32s3-display-2.8-lcd"
```

Output:

- build/merged-binary.bin — file flash chính (~10.4 MB).
- releases/v2.2.6_freenove-esp32s3-display-2.8-lcd.zip — zip release.
- build/xiaozhi.elf — debug symbols cho stack trace.

Flash (Flash Download Tool)

1. Mở Flash_download.zip trong kit, chạy flash_download_tool_3.9.9_R2.exe.
2. Chip: **ESP32-S3**, WorkMode: **Develop**, LoadMode: **UART**.
3. File: build/merged-binary.bin, Address: 0x0, tick checkbox đầu dòng.
4. Tick DoNotChgBin, FLASH SIZE 128Mbit, SPI MODE DIO, SPI SPEED 80MHz.
5. COM port (vd. COM9), BAUD 460800.
6. Click **ERASE** trước (tránh rác từ flash cũ), sau đó **START**.
7. Đợi đến khi báo **FINISH** xanh.

Manual bootloader entry (nếu fail)

1. Giữ nút **BOOT (IO0)** trên board.
2. Vẫn giữ BOOT → nhấn **RST/EN** 1 cái rồi thả.
3. Thả BOOT.
4. Click **START** trong FDT.

Monitor serial sau khi flash

```
$p = New-Object System.IO.Ports.SerialPort 'COM9',115200,'None',8,'One'
$p.Open(); while ($true) { try { Write-Host $p.ReadLine() } catch {} }
```

Hoặc dùng idf.py monitor:

```
cmd.exe /c "idf_env.bat idf.py -p COM9 monitor"
```

(Ctrl+] để thoát)

Plan tổng — chia phase

c|Y|c|Y **Phase & Việc & Thời gian & Phụ thuộc**

& Đọc tài liệu này, hiểu kiến trúc & 1h & –

1 & Xác nhận: WS server URL + servo wiring & — & –

2 & Code servo_controller.cc/h & 30p & 1

3 & Sửa InitializeTools() đăng ký 3 MCP tool & 20p & 2

4 & Sửa ota.cc hoặc Kconfig cho server custom & 30p & 1

5 & Build + flash + smoke test (LCD lên, MQTT/WS connect) & 15p & 3,4

6 & Test chain end-to-end: nói "vẫy tay" → robot vẫy & 30p & 5

7 & Optimize: AEC, latency, watchdog, persona prompt & lặp & 6

Câu hỏi cần xác nhận trước khi code

1. Server WebSocket có endpoint /ota chưa, hay chỉ raw WebSocket?
2. URL server: wss://example.com/ws hay IP nội bộ ws://192.168.x.x:port?
3. Authentication: server check token Bearer không?
4. Servo cấp nguồn riêng nguồn nào? (2S Li-ion + buck 6V là tối ưu)
5. GPIO 14, 21, 2 layout phần cứng OK chứ?
6. Server có hỗ trợ MCP tools/list & tools/call JSON-RPC chưa? Nếu chưa, người làm server cần thêm.

Glossary — từ điển

Analog-to-Digital Converter — đọc voltage analog (vd. điện áp pin).

Acoustic Echo Cancellation — khử tiếng vọng (loa vào mic).

Audio Front-End — pipeline xử lý audio gồm AEC + NS + VAD + WakeNet.

Automatic Speech Recognition — giọng → text.

Bluetooth Low Energy.

Code chạy đầu tiên khi cấp điện, load app từ flash.

Chip mã hóa/giải mã audio analog ↔ digital. Board dùng ES8311.

Framework chính thức Espressif lập trình ESP32.

Real-Time OS chạy trên ESP32, scheduler đa task.

Inter-Integrated Circuit — bus 2 dây config codec/touch.

Inter-IC Sound — bus truyền audio digital.

Low-Dropout Regulator — chip ổn áp.

LED Controller peripheral của ESP32, dùng làm PWM.

Light and Versatile Graphics Library — UI framework cho LCD.

Model Context Protocol — chuẩn AI gọi tool.

Microcontroller Unit.

Message Queuing Telemetry Transport — pub/sub broker protocol.

Non-Volatile Storage — KV store nhỏ trên flash, persist qua reboot.

Audio codec lossy (giống MP3 nhưng tốt hơn cho voice).

Over-The-Air update — update firmware qua mạng.

Power Amplifier — khuếch đại ra loa.

Bảng phân bổ vùng flash.

Pseudo-SRAM — RAM mở rộng ngoài chip (board có 8MB octal).

Pulse Width Modulation — điều khiển servo bằng độ rộng xung.

Quad SPI — bus 4-line tốc độ cao cho flash.

System-on-Chip — vi điều khiển tích hợp đủ Wi-Fi, BLE, RAM.

Serial Peripheral Interface — bus 4-dây cho LCD.

Speech-To-Text = ASR.

GPIO mà trạng thái lúc reset quyết định boot mode.

Text-To-Speech — text → giọng.

Universal Asynchronous Receiver-Transmitter — serial truyền tin.

Native USB của ESP32-S3, không cần chip CP210x/CH340.

Voice Activity Detection — phát hiện có người nói.

Mạng nơ-ron offline phát hiện wake word ("Hi, ESP").

Protocol giữ kết nối TCP 2 chiều mở vĩnh viễn.

Tài liệu tham khảo

1. Xiaozhi-esp32 GitHub: <https://github.com/78/xiaozhi-esp32>
2. ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/en/v5.5.4/>
3. ESP32-S3 Technical Reference:
https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
4. Kit ES3C28P (LCD wiki): https://www.lcdwiki.com/2.8inch_ESP32-S3_Display
5. Anthropic MCP Spec: <https://modelcontextprotocol.io/>
6. LVGL Documentation: <https://docs.lvgl.io/>
7. ESP-SR (AFE/WakeNet): <https://github.com/espressif/esp-sr>

*Tài liệu này được tạo tự động bởi Claude (Anthropic). Cập nhật lần cuối: 2026-05-10.
Liên hệ chỉnh sửa: re-prompt Claude với yêu cầu cụ thể.*